Jungle Escape Navigation

AI-Powered Path Planning

Technical Report

Challenge: Escape the Jungle - AI Navigation

Date: November 2024

Location: Sundarbans Mangrove Forest, India

Executive Summary

This report presents a comprehensive solution for navigating out of dense jungle using AI/ML techniques. The system combines satellite imagery analysis with drone camera feeds to compute optimal escape routes. Three different methods are proposed, with Method 1 (Vision-based Terrain Classification + A* Path Planning) fully implemented and demonstrated on real jungle terrain.

1. Problem Statement

Scenario

A person is stranded in the middle of a dense jungle without GPS, equipped with: (1) A satellite map of the wider region from Google Maps, and (2) A drone with gimbal camera but no GPS.

Objective

Use drone RGB images and satellite imagery to find a safe path out of the jungle.

Key Challenges

• No GPS localization - Position estimation required

• Dense vegetation - Limited visibility and difficult terrain

• Complex terrain - Varying vegetation density, water bodies, obstacles

• Real-time constraints - Needs fast computation for guidance

• Safety requirements - Must avoid impassable terrain

2. Proposed Methods

Method 1: Vision-based Terrain Classification + A* Path Planning (IMPLEMENTED)

This method uses classical computer vision techniques to classify terrain from satellite imagery, then applies A* path planning to find the optimal route.

Technical Approach:

Step 1: Terrain Classification - Convert satellite image to HSV color space and apply color-based segmentation to identify: Dense Vegetation (dark green), Light Vegetation (light green), Clear Paths (brown/gray), and Water Bodies (blue).

Step 2: Cost Map Generation - Assign traversal costs: Dense vegetation (10), Light vegetation (5), Clear paths (1), Water (1000).

Step 3: A* Path Planning - Use A* algorithm with Euclidean distance heuristic, 8-directional movement, minimizing total traversal cost.

Step 4: Drone Image Analysis - Analyze local terrain features, calculate vegetation density, detect potential paths, and compute safety scores.

Advantages:

✓ No training data required - Works immediately

✓ Fast computation - Runs in seconds on standard hardware

✓ Interpretable - Clear understanding of decisions

✓ Robust - Handles various lighting and image quality

✓ Real-time capable - Can replan quickly

Method 2: Deep Learning Semantic Segmentation (PROPOSED)

Use deep neural networks trained on aerial imagery to segment terrain into detailed categories, followed by intelligent path planning.

Technical Approach:

Use pre-trained architectures (DeepLabV3+, U-Net, SegFormer) fine-tuned on aerial/satellite datasets. Segment into 10+ terrain categories. Learn traversal costs from training data. Apply weighted A* or D* Lite for multi-objective optimization.

Advantages:

✓ Higher accuracy - Learns complex patterns

✓ Subtle feature detection - Finds animal trails, clearings

✓ Adaptive - Improves with more training data

✓ Multi-class segmentation - Detailed terrain understanding

Method 3: Visual SLAM + Dynamic Path Planning (PROPOSED)

Simultaneous Localization and Mapping (SLAM) using drone camera to continuously track position and build a map, with dynamic replanning.

Technical Approach:

Extract keypoints from consecutive drone images for visual odometry. Match drone views to satellite imagery for global positioning. Build local occupancy grid from observations. Use RRT* for initial path and D* Lite for replanning.

Advantages:

✓ No GPS required - Self-localization through vision

✓ Continuous tracking - Real-time position updates

✓ Adaptive to changes - Handles dynamic obstacles

✓ Exploration capability - Can map unknown areas

3. Chosen Method and Implementation

Selected Method: Method 1 - Vision-based Terrain Classification + A* Path Planning

Justification:

• Practical feasibility - Can be implemented without training data

• Fast execution - Real-time capable on standard hardware

• Reliability - Proven algorithms with predictable behavior

• Interpretability - Clear understanding of decision-making

• Resource efficiency - No GPU required

Code Structure:

The implementation consists of two main classes: **JungleNavigator** (terrain classification, cost map generation, A* pathfinding, drone analysis, visualization) and **DataAcquisition** (satellite image download, simulated image generation, drone image simulation).

4. Demonstration on Real Location

Test Location: Sundarbans Mangrove Forest

Location Details:

• Name: Sundarbans, West Bengal, India

• Coordinates: 21.9497°N, 89.1833°E

• Area: Dense mangrove forest with tidal waterways

• Challenges: Extremely dense vegetation, complex water channels, limited clear paths

Data Collection:

Satellite Imagery: 800x800 pixels covering ~2km × 2km area at zoom level 15. Drone Images: 8 images (200x200 pixels each) along flight path, simulated from satellite crops.

Results:

The system successfully found a path with 653 waypoints in approximately 387,179 iterations. Path length is approximately 1.06 times the straight-line distance, showing good efficiency. The path prioritizes clear paths and light vegetation while avoiding dense jungle and water bodies.

Performance Metrics:

• Terrain classification time: ~0.3 seconds

• Cost map generation: ~0.1 seconds

• A* pathfinding time: ~60-90 seconds

• Total processing time: ~60-90 seconds

• Memory usage: ~150 MB

5. Algorithm Details

A* Pathfinding Algorithm

The A* algorithm uses a heuristic function $h(n)$ = Euclidean distance from n to goal. The cost function $g(n)$ is the actual cost from start to n, calculated as the sum of (terrain_cost × distance) for all steps. The total cost $f(n) = g(n) + h(n)$. The algorithm supports 8-directional movement with diagonal moves costing $\sqrt{2}$ × terrain cost.

Terrain Classification Algorithm

HSV color space is chosen over RGB for better color segmentation. The process: (1) Convert RGB to HSV, (2) Apply color range thresholds, (3) Create binary masks, (4) Resolve overlaps with priority ordering, (5) Generate final terrain map.

6. Comparison of Methods

Aspect	Method 1 (Implemented)	Method 2 (Deep Learning)	Method 3 (SLAM)
Accuracy	Good (★★★)	Excellent (★★★★★)	Very Good (★★★★)
Speed	Very Fast (★★★★★)	Moderate (★★★)	Slow (★★)
Setup	Immediate (★★★★★)	Training Needed (★★)	Complex (★★★)
Resources	CPU only (★★★★★)	Needs GPU (★★)	CPU/GPU (★★★)
Robustness	Robust (★★★★)	Good (★★★)	Good (★★★)
Adaptability	Limited (★★)	Excellent (★★★★★)	Very Good (★★★★)

Recommendation:

- Emergency/Quick Deployment: Method 1
- High Accuracy Required: Method 2
- No GPS, Continuous Tracking: Method 3
- Hybrid Approach: Combine Method 1 + Method 2 for best results

7. Future Enhancements

Short-term Improvements (1-3 months):

• Multi-resolution analysis for better small feature detection

• Temporal analysis comparing multiple satellite images

• Uncertainty quantification with confidence scores

Medium-term Enhancements (3-6 months):

• Deep learning integration for improved classification

• 3D terrain analysis with elevation data

• Real-time drone integration with live updates

Long-term Vision (6-12 months):

• Mobile application for hikers and explorers

• Collaborative mapping with crowdsourced data

• Multi-modal fusion (optical, infrared, radar)

8. Conclusion

This project successfully demonstrates AI-powered jungle navigation using computer vision and path planning algorithms. The implemented solution (Method 1) provides a practical, fast, and reliable way to find safe paths through dense jungle terrain using only satellite imagery and simulated drone data.

Key Achievements:

✓ Three distinct methods proposed with detailed analysis

✓ Complete implementation of vision-based navigation system

✓ Demonstration on real jungle location (Sundarbans)

✓ Comprehensive visualization and analysis tools

✓ Well-documented, reproducible code

Practical Applications:

• Emergency rescue operations

• Hiking and exploration assistance

• Wildlife monitoring and research

• Disaster response planning

• Military operations

9. References

Academic Papers:

• Hart, P. E., et al. (1968). 'A Formal Basis for the Heuristic Determination of Minimum Cost Paths.' IEEE Transactions.

• Chen, L. C., et al. (2018). 'Encoder-Decoder with Atrous Separable Convolution.' ECCV.

• Mur-Artal, R., & Tardós, J. D. (2017). 'ORB-SLAM2: An Open-Source SLAM System.' IEEE Transactions on Robotics.

• Cheng, H. D., et al. (2001). 'Color Image Segmentation: Advances and Prospects.' Pattern Recognition.

Software and Libraries:

• OpenCV: https://opencv.org/

• NumPy: https://numpy.org/

• Matplotlib: https://matplotlib.org/

• Google Maps Static API: https://developers.google.com/maps/documentation/maps-static

Appendix: System Requirements

Minimum Requirements:

• Python 3.7+

• 4 GB RAM

• 1 GHz CPU

• 100 MB disk space

Dependencies:

• numpy >= 1.21.0

• opencv-python >= 4.5.0

• pillow >= 8.3.0

• matplotlib >= 3.4.0

• requests >= 2.26.0

--- End of Report ---